

活动社交项目技术说明文档

- 活动社交项目技术说明文档
 - 1. 文档目的
 - 2. 项目概述
 - 3. 前端使用了什么技术
 - 4. 后端使用了什么技术
 - 5. 数据库和 SQL 部分怎么做
 - 6. 项目是怎么部署在 AWS 上的
 - 7. 现在团队成员应该如何分工
 - 7.1 前端同学如何开始
 - 7.2 后端同学如何开始
 - 7.3 SQL / 数据库同学如何开始
 - 8. 本地开发应该从哪里开始
 - 第一步：拉取代码
 - 第二步：安装依赖
 - 第三步：准备数据库
 - 第四步：访问项目
 - 9. 如果要继续同步到服务器，应该怎么做
 - 10. 当前阶段的建议
 - 11. 结论

活动社交项目技术说明文档

1. 文档目的

本文档用于回答当前项目在实现阶段的几个核心问题，包括前端和后端分别采用什么技术、数据库如何组织、项目如何部署在 AWS 上，以及如果团队成员分别负责前端、后端或 SQL，应该如何进入项目并开始协作开发。本文档适合作为组内技术说明、答辩准备材料或阶段性开发说明使用。

2. 项目概述

本项目是一个面向课程交付场景的活动社交类 Web 应用，目标是实现“活动发现、活动收藏、免费报名、管理员审核与外部活动同步”的最小可交付版本。项目采用前后端分离思路，但以单仓库形式统一管理，既方便本地开发，也方便在服务器上统一部署和更新。

当前项目已经完成的核心能力包括：用户注册与登录、首页活动展示、活动搜索与筛选、活动详情查看、收藏与取消收藏、免费 RSVP 报名、我的收藏、我的报名记录、管理员活动状态管理，以及外部活动同步与同步日志查看。

3. 前端使用了什么技术

前端使用的是 Next.js + React + TypeScript。

其中，Next.js 负责页面路由、构建与生产环境输出，React 负责页面组件与交互逻辑，TypeScript 用于保证前端代码的类型安全和可维护性。前端代码位于 `apps/web` 目录下，页面主要集中在 `apps/web/app`，通用组件位于 `apps/web/components`，接口访问封装位于 `apps/web/lib`。

从页面结构来看，前端当前已经包含以下模块：

- 首页
- 搜索页
- 登录页
- 注册页
- 活动详情页
- 我的收藏页
- 我的报名页
- 管理员后台页

前端现在采用移动端优先的响应式 Web 设计，因此同一套代码既可以在电脑浏览器访问，也可以在手机浏览器中访问。

4. 后端使用了什么技术

后端使用的是 Node.js + Express + TypeScript。

Node.js 作为运行时环境，Express 负责 REST API 路由组织与请求处理，TypeScript 用于保证接口层和业务逻辑层的类型约束。后端代码位于 `apps/api` 目录下，其中：

- `src/routes` 用于定义接口路由
- `src/middleware` 用于处理鉴权与权限拦截
- `src/services` 用于处理外部活动同步等业务逻辑
- `src/lib` 用于处理 Prisma、JWT 等基础能力

后端当前已实现的主要接口包括：

- 用户注册接口
- 用户登录接口
- 活动列表接口
- 活动详情接口
- 收藏与取消收藏接口
- 免费报名接口
- 我的收藏接口
- 我的报名接口
- 管理员活动管理接口
- 外部同步接口
- 同步日志接口

系统鉴权方式采用 JWT。用户登录成功后，后端会生成 token，前端在需要访问受保护接口时通过 `Authorization` 请求头携带该 token。

5. 数据库和 SQL 部分怎么做

数据库使用的是 MySQL/MariaDB，ORM 使用的是 Prisma。

项目并不是手写大量分散 SQL 文件，而是通过 `prisma/schema.prisma` 集中定义数据模型，然后通过 Prisma migration 生成数据库迁移脚本。这种方式比手工维护零散 SQL 更清晰，也更适合多人协作。

当前数据库模型主要包括以下几类数据：

- 账号表 `Account`
- 角色表 `Role`
- 用户角色映射表 `AccountRole`
- 活动表 `Event`
- 收藏表 `Favourite`
- 报名表 `Booking`
- 外部接口配置表 `ApiIntegration`
- 同步日志表 `SyncLog`

如果队友负责 SQL 或数据库部分，最重要的入口文件就是：

- `prisma/schema.prisma`

常规工作流程如下：

1. 修改 `schema.prisma`
2. 运行 migration 命令生成迁移
3. 同步数据库结构

4. 如有需要，更新 seed 数据

常用命令如下：

```
pnpm db:generate
pnpm db:migrate --name your_change
pnpm db:seed
```

因此，SQL 同学在这个项目里更准确的职责不是“只写 SQL 文件”，而是“负责数据库模型、关系设计、迁移维护与测试数据准备”。

6. 项目是怎么部署在 AWS 上的

项目当前部署在一台 AWS EC2 Ubuntu 服务器上。

部署结构采用的是单机部署方案，整体架构如下：

- AWS EC2 服务器负责承载整个应用
- Nginx 作为反向代理
- Next.js 前端服务运行在 3000 端口
- Express 后端服务运行在 4000 端口
- MariaDB 数据库运行在服务器本机 3306 端口
- PM2 用于管理前后端进程，保证服务常驻运行

实际转发关系如下：

- 浏览器访问 `/` 时，由 Nginx 转发到前端服务
- 浏览器访问 `/api/...` 时，由 Nginx 转发到后端服务

这样做的优点是：

- 用户只需要访问一个域名或公网地址
- 前后端在部署层面被统一入口管理
- PM2 可以保证进程异常退出后自动恢复
- 后续更新时只需 `git pull + build + restart`

当前服务器部署流程大致如下：

1. 在 EC2 上安装 Node.js、pnpm、PM2、Nginx、MariaDB
2. 从 GitHub 拉取项目代码
3. 配置 `.env`
4. 执行数据库迁移和种子数据初始化
5. 构建前端和后端
6. 使用 PM2 启动前端和后端

7. 使用 Nginx 做反向代理

7. 现在团队成员应该如何分工

如果队友现在要进入项目写代码，最清晰的方式是按模块分工，而不是所有人都同时从仓库根目录随意修改。

7.1 前端同学如何开始

前端同学主要进入 `apps/web` 目录开发。

建议优先关注：

- `apps/web/app`
- `apps/web/components`
- `apps/web/lib`

前端同学主要负责：

- 页面结构和视觉表现
- 表单交互
- 页面跳转
- 登录态展示
- 用户体验与移动端适配
- 页面文案与展示逻辑

如果只想启动前端本地开发，可以执行：

```
pnpm --filter web dev
```

7.2 后端同学如何开始

后端同学主要进入 `apps/api` 目录开发。

建议优先关注：

- `apps/api/src/routes`
- `apps/api/src/services`
- `apps/api/src/middleware`
- `apps/api/src/lib`

后端同学主要负责：

- 新接口开发
- 参数校验

- 鉴权逻辑
- 管理员权限控制
- 外部 API 集成
- 业务逻辑扩展

如果只想启动后端本地开发，可以执行：

```
pnpm --filter api dev
```

7.3 SQL / 数据库同学如何开始

SQL 或数据库同学主要进入 `prisma` 目录工作。

建议优先关注：

- `prisma/schema.prisma`
- `prisma/migrations`
- `prisma/seed.ts`

这部分同学主要负责：

- 设计表结构
- 维护实体关系
- 新增字段与约束
- 生成迁移
- 准备初始数据或测试数据

8. 本地开发应该从哪里开始

如果一个新成员第一次接触项目，建议按以下顺序开始：

第一步：拉取代码

```
git clone https://github.com/zhaoruizhe81-bot/eventsfinder-mvp.git
cd eventsfinder-mvp
```

第二步：安装依赖

```
pnpm install
```

第三步：准备数据库

如果本机有 MySQL，可直接使用项目提供的本地脚本：

```
pnpm local:start
```

这个脚本会自动：

- 创建本地数据库
- 执行 migration
- 写入 seed 数据
- 启动前后端服务

第四步：访问项目

本地运行后默认访问地址为：

- 前端：`http://localhost:3000`
- 后端：`http://localhost:4000`

9. 如果要继续同步到服务器，应该怎么做

当前项目已经放在 GitHub 公开仓库中，因此服务器不需要额外配置 GitHub 私钥或 token，就可以直接同步代码。

推荐的服务器更新流程如下：

```
cd /home/ubuntu/eventsfinder-mvp
git pull
pnpm install --frozen-lockfile
pnpm db:deploy
pnpm build
pm2 restart eventsfinder-api
pm2 restart eventsfinder-web
```

如果这次只是前端页面或前端文案更新，也可以只重建前端并重启前端服务。

10. 当前阶段的建议

从团队协作角度看，现在最重要的不是继续盲目加新功能，而是先把三件事稳定下来：

- 页面英文文案统一

- 前后端联调路径稳定
- 数据库结构不要频繁随意变更

建议你们小组当前的协作方式是：

- 一人主前端页面与交互
- 一人主后端接口与业务逻辑
- 一人主数据库模型和测试数据
- 其他成员辅助文档、测试、部署与答辩材料准备

这样比所有人一起改同一块代码要高效得多，也更适合课程项目在短时间内完成稳定交付。

11. 结论

综上，本项目的实现方式可以总结为：

- 前端：Next.js + React + TypeScript
- 后端：Node.js + Express + TypeScript
- 数据库：MySQL/MariaDB + Prisma
- 部署：AWS EC2 + Nginx + PM2 + MariaDB
- 协作：GitHub 仓库统一管理，本地开发完成后推送，服务器通过 `git pull` 同步更新

如果队友现在想参与项目开发，最直接的入口就是根据自己的职责进入对应目录：

- 前端看 `apps/web`
- 后端看 `apps/api`
- 数据库看 `prisma`

这样就能比较清晰地知道“从哪里开始写”，也方便后续把开发过程整理成答辩文档、课程报告或阶段性 PDF 材料。